

微信 AI 记忆库 wechat-ai-memory

把散落在微信里的对话，整理成 Agent 能读、能检索、能引用的本地记忆档案

仓库 [ikevss/LOGO127 · wechat-ai-memory](#) 版本 **v0.3.7-alpha** 协议 **MIT**

平台 **Windows 10/11 × 微信 4.x**

技术栈 **Python 3.11+ · PySide6 · Frida · ReportLab · faster-whisper** 学习日期 **2026-10-03**

一句话 微信 AI 记忆库是一个**本地优先的开源 Windows 工具**：直接读取本机微信 4.x 的聊天记录（也支持纯 JSON 数据源），按**会话 / 日期 / 关键词**筛选后，导出成 **PDF + Markdown + 结构化 JSON** 三件套——目的不是“备份聊天”，而是给 AI Agent 准备一份**可引用、可追溯的个人上下文记忆**。

疑问即目录：10 个问题讲透它 Questions First —— 工科毕业生的视角

Q1 这是什么？解决什么问题？

Agent 时代有个尴尬现实：**模型能不能帮上忙，取决于它懂不懂“事情的来龙去脉”**。项目决策、需求变更、关键承诺……这些上下文不在项目管理工具里，而是**散落在微信私聊和群聊里**。你给模型几句临时描述，它只能靠猜。

这个项目的定位一句话：**微信 = 你的最大上下文仓库，但它对 Agent 是个黑盒 → 这个工具把黑盒变成 Agent 可读的档案**。导出的 JSON 里每条消息都带 `id`、`sender`、`timestamp`、`reply_to`，模型引用时能精确回溯到原文——这就是“可追溯”的含义。

Q2 为什么需要它？没有它之前人们怎么做？

现有的路各有毛病：

老办法	痛点
付费闭源导出工具	要花钱，且聊天记录上传到谁的服务器 无法独立审查 ——心里没底
手工复制粘贴 / 截图	几百条消息不可持续，截图里的文字模型没法直接检索
口头向 Agent 复述背景	转述必有遗漏和失真，承诺、结论这类细节最容易丢

它的差异化主张就三条：**开源免费 (MIT)**、**本地优先 (无遥测无远程 API)**、**数据不上传**。当前版本先把“可靠的提取与归档”这一层做扎实，语义检索 / RAG / MCP 都排在路线图上（见 Q9）。

Q3 它是怎么做到的？整体架构长什么样？

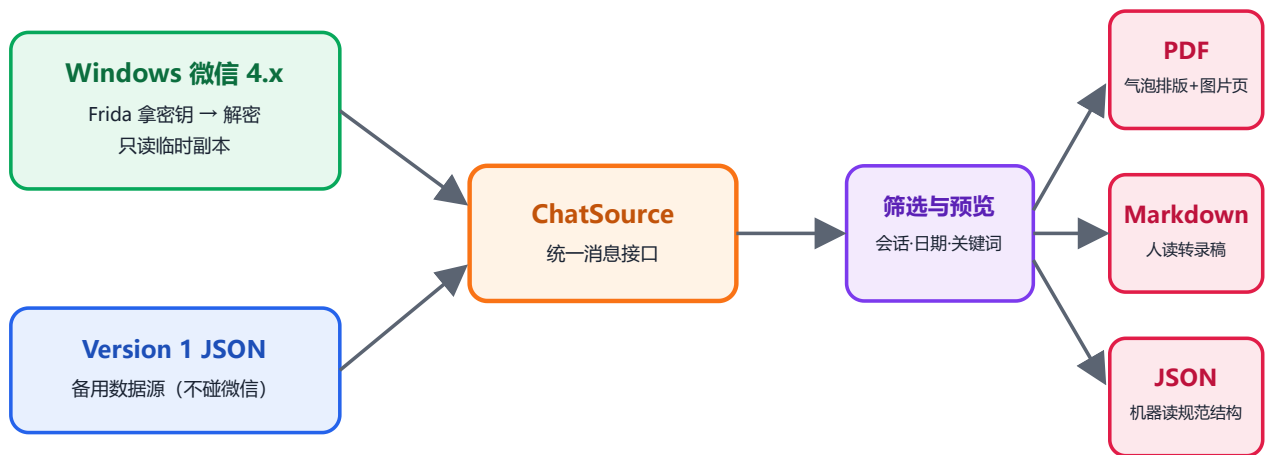


图 1 · 双数据源汇入统一 ChatSource 接口，筛选后分流三格式导出

架构上最值得学的一手是**适配器隔离**：微信数据适配器（`sources/wechat4_*.py` 六个模块）和渲染、导出模块完全解耦，两者只通过 `ChatSource` 抽象接口对话。就算哪天微信改了内部格式把适配层打挂，**JSON 数据源和整条导出管线照样独立存活**——这也是我们本批次实验敢跑的前提（只用 JSON 入口，绝不碰真实微信）。

Q4 微信的数据到底是怎么被读出来的？

微信 4.x 的本地数据库是 **SQLCipher 整库加密**的，密钥只存在于运行中的微信进程内存里。这个项目的读取流程：

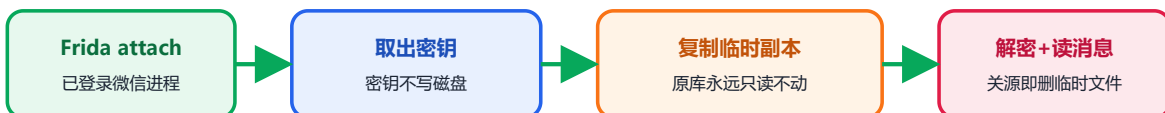


图 2 · “进程内取钥 → 副本上解密”路线：原数据库文件从头到尾不被修改

对比已学概念 这与 vault 里 **[[CleanRoom只读读取器]]**（wechat-intelligence-hub 的路线）是同一目标的两条技术路线：CleanRoom 坚持不碰目标进程，密钥材料由用户本人提供（授权问题）；本项目走 **Frida 动态注入拿密钥**——自动化程度高（用户零操作），但代价是要注入微信进程、被安全软件盯上、且微信升级随时可能打挂。两条路线的隐私底线一致：**只读、密钥不落盘、临时文件用完即删**。

Q5 导出的 JSON 到底长什么样？Agent 怎么“读”？

所谓“Agent 可读”，核心就是这份**规范化 (normalized) JSON**。我们用官方虚构样例 `demo_chat.json` 实测导出，结构如下：

```
{
  "version": 1,
  "conversations": [{
    "id": "eco-project", "name": "ECO 项目讨论群", "kind": "group",
    "messages": [{
      "id": "m004", // 稳定消息 ID (路线图 v0.4 会强化去重)
      "sender": "张三", "timestamp": "2026-08-21T09:22:00",
      "type": "text", // text / image / file / voice / system
      "content": "图里有两个信息需要复核：.....",
      "is_outgoing": false, // 谁在说话：本人 or 对方
      "reply_to": "m003" // 引用回复 → 消息间的引用链
    }, ...]
  }]
}
```

三个设计点：① `is_outgoing` **区分“我说的”和“别人说的”**——对 Agent 理解立场至关重要；② `reply_to` **保留消息引用链**——“张三那句话是针对哪张图说的”不再是悬念；③ `type` **五类全覆盖**，图片/文件导出后指向 `_assets` 目录的相对路径。这其实是给 LLM 喂上下文时最省 token 又最不丢信息粒度的中间格式。

Q6 语音和图片这些“非文字消息”怎么处理？

🔊 语音 (SILK 格式)

微信语音是 **SILK** 编码（微信自家的语音编解码格式，普通播放器打不开）。项目用 `silk-python` 解码，再交给 **faster-whisper**（本地运行的开源转写模型，首次需联网下载约 500 MB 的 small 模型）转成文字并**缓存**——转写全程在本机，语音不上传。

🖼️ 图片 (V2 / WXGF)

微信 4.x 的图片是自有封装格式 (WXGF)，需要“**原图恢复**”；恢复后再做 **EXIF 方向修正**（按拍摄方向字段转正，避免 PDF 里横躺），输出预览图 + **300 DPI 高清独立页**。实测 PDF 里图片页还带“来源会话/发送者/时间”的元数据头。

要点 语音转写缓存和模型都存 `%LOCALAPPDATA%\WeChatAIMemory`，首次下载模型后可完全离线使用——这延续了“本地优先”的承诺。

Q7 三方对比：它 vs 微信情报库 vs Memmy，到底什么关系？

流水线已经学过两个同域项目，必须摆在一起看清楚。一句话定位：**wechat-ai-memory 是进料口**，**wechat-intelligence-hub 是加工车间**，**Memmy 是仓库+检索台**——三者其实可以串成一条完整的个人记忆流水线。

	微信 AI 记忆库（本项目）	微信个人情报库 wechat-intelligence-hub	Memmy 记忆中枢
解决什么	微信记录 → Agent 可读档案（搬数据）	微信记录 → 情报日报+行动信号（出情报）	跨工具记忆的存储与检索（管记忆）
读微信方式	Frida 注入取钥 + 临时副本解密	CleanRoom 不碰进程，本人提供密钥材料	不读微信（吃各 Agent 的会话记录）
加工方式	确定性筛选（会话/日期/关键词）+ 排版导出	[[确定性信号引擎]] 规则打信号、附证据原句	四层记忆（轨迹→策略→世界模型→技能）+ 混合检索
输出	PDF / MD / JSON 三件套	HTML 情报日报	统一检索 API 给任意 Agent
Agent 接口	暂无（MCP 在 v0.8 路线图）	无（人看报）	已有 （跨工具共享检索）
成熟度	v0.3.7-alpha，数据层扎实	概念验证完善	全本地闭环可用



三者拼起来 = 完整的个人记忆流水线：进料 → 加工 → 存储/检索/供给

图 3 · 三个项目在“个人记忆流水线”上的分工（互链见 vault 概念笔记）

关键差异点在**消费端**：情报库出的是给人看的日报，Memmy 出的是给任何 Agent 用的检索接口；而本项目现在只出到“档案文件”，**它变成真正 Agent 记忆的最后一公里（MCP 只读查询）排在路线图 v0.8**。从这个角度看，它当前的 JSON 输出已经可以直接被 Memmy 这类系统当语料吃进去。

Q8 隐私红线是怎么设计的？

数据行为	是否发生	实现机制
上传聊天内容 / 图片 / 导出文件	否	无遥测、无远程 API、无 AI 云请求
写密钥到磁盘	否	Frida 内存取钥，进程结束即消失
修改原始微信数据库	否	只解密"临时副本"，原库只读
发 / 撤 / 改微信消息	否	纯读取，无写通道
临时文件残留	否	数据源关闭时删除临时库与恢复图片

但要注意 ① 未做代码签名，SmartScreen 会拦；Frida 注入行为可能触发安全软件告警——**只从官方 Release 下载**。② "数据不上传"承诺目前靠**自律 + 开源可审查**，alpha 阶段没有第三方审计背书。③ 本批次实验的纪律就是这条红线的实践：**只用官方虚构 demo 和自造样例数据，绝不连接真实微信库**。

Q9 现在能干什么、不能干什么？

已经能（当前能力）：

本机微信读取（只读副本）

会话/日期/关键词三重筛选

文本·图片·文件·语音·系统消息

语音本地转写（whisper）

PDF/MD/JSON 三格式导出

GUI + CLI + JSON 备用入口

还不能（明确的边界）：AI 问答、RAG 语义检索、OCR、向量索引——**它现在是"可信数据层"，不是"智能层"**。路线图：v0.4 持久记忆库+增量同步 → v0.5 SQLite FTS5 全文检索 → v0.6 结构化实体（人物/项目/决策）→ v0.7 带引用的本地 AI 问答 → v0.8 只读 MCP 接口。也就是说，"变成真 Agent 记忆"是它自己认定的终局，只是当下先把地基打牢。

Q10 怎么跑起来？普通用户和开发者各一条路

普通用户（连真微信）

1. Release 下载 `WeChatAIMemory-Portable.exe`（免安装，内含全部运行库）
2. 登录微信 4.x → 点"连接微信"→ 微信重启后 5 分钟内确认
3. 选联系人 / 日期 / 关键词 → 生成记忆档案

开发者（只跑 JSON 入口，本次实验路线）

1. `git clone` → 建 venv
2. 装依赖（JSON 入口最小集：Pillow + reportlab + zstandard + pycryptodome）
3. `python -m wechat_context_exporter --source demo_chat.json ...`

实测坑 包的 `__init__.py` 会连带 import 微信适配层 → **即使你只想用 JSON 入口，也必须装 zstandard 和 pycryptodome**，否则 `ModuleNotFoundError`。Frida 和 faster-whisper 这两个大件倒是可以不装（用不到就不触发）。

这个项目给我什么：能力清单 What You Get —— 结合当前水平的具体用途



5 个马上能用的场景

1. **给 Agent 补背景**：开新会话前，把某项目相关的群聊按日期筛出来导出 JSON/MD，直接贴给 Claude——比口头复述完整得多，且每条可回溯原文。
2. **自造对话语料**：Version 1 JSON 格式简单（本次实验已验证），可以**手工构造虚构对话**测试任何“聊天记录分析”想法，零隐私风险。
3. **项目复盘档案**：把一个项目周期内的群聊导成带引用链的 PDF，作为结项归档材料。
4. **为记忆流水线备料**：将来把导出的 JSON 喂给 Memmy 类系统做长期记忆——进料口（本项目）×仓库（Memmy）组队。
5. **学架构范式**：适配器隔离 + 统一 ChatSource 抽象 + 备用数据源保活，是“依赖易变外部系统”时极值得抄的工程结构。

成就感实验：JSON 备用入口全链路（隐私红线内）

All commands verified on Windows 11 · Python 3.14 · 2026-10-03

红线声明 本实验全程**未连接真实微信**：数据源只用官方虚构样例 `demo_chat.json`（ECO 项目讨论群）和自造样例 `my_chat.json`（记忆库选型小组，人物情节均为虚构）。

1 跑通最小环境，列出样例里的会话

操作（venv + 最小依赖，Pillow/reportlab/zstandard/pycryptodome 四件）：

```
python -m venv .venv
.venv\Scripts\pip install Pillow reportlab zstandard pycryptodome
$env:PYTHONPATH="repo\src" # 或 pip install -e repo
.venv\Scripts\python -m wechat_context_exporter --source repo\examples\demo_chat.json
```

目的验证环境与 CLI 数据源识别 | **实现**不装 Frida/whisper 大件也能驱动整条导出管线 | **检测**

✅ **预期结果（实测）**：列出 2 个会话——`eco-project ECO 项目讨论群 8 messages`、`mentor-chat 导师 2 messages`

2 三格式导出：一份档案，三种读者

```
python -m wechat_context_exporter --source repo\examples\demo_chat.json `
  --conversation eco-project --start 2026-08-21 --end 2026-08-22 `
  --output demo-context.pdf --markdown demo-context.md --json demo-context.json
```

目的一次生成 PDF（人看） / Markdown（转录） / JSON（机器读） | **实现**日期范围筛选 + 气泡排版 + 图片独立页 | **检测**

✅ **预期结果（实测）**：Created demo-context.pdf (2 pages, 8 messages, 1 image pages)；PDF 目检：微信风格左右气泡、引用回复框（Reply to m003）、系统消息居中灰条、训练曲线图带 300DPI 附件页（元数据头：来源/发送者/时间）

3 关键词筛选：从 8 条里精确捞出 1 条

```
python -m wechat_context_exporter --source repo\examples\demo_chat.json `
  --conversation eco-project --query 过拟合 --output demo-query.pdf --json demo-query.json
```

目的验证 `--query` 按发送者/内容/类型全文匹配 | **检测**

✅ **预期结果（实测）**：1 messages，JSON 中唯一命中 m002（李四".....可能出现了过拟合"，is_outgoing: true）

4 自造样例数据：验证"我的格式我说了算"

操作：手写 `my_chat.json`（虚构"记忆库选型小组"群聊 7 条 + "导师"私聊 1 条，含 reply_to 引用链、system 消息、图片附件），喂进同一管线。自造 PNG 附件用 Pillow 画一张。

目的验证数据主权——JSON 备用入口不依赖微信也能出完整档案 | **检测**

✅ **预期结果（实测）**：Created my-context.pdf (1 pages, 5 messages)；2026-09-01 当天 5 条（n001~n005）日期过滤精确；reply_to 链 n003→n002 保留

踩坑实录 ① `kind` 只认 `direct / group`，写 `private` 直接报错 `'private' is not a valid ConversationKind`（错误信息清晰，护栏合格）；② JSON 字符串里嵌 ASCII 双引号必须转义（改用「」最省事），否则 `Expecting ',' delimiter`。

5 体检"Agent 可读性": 规范化 JSON 的字段契约

操作: 用 Python 检查导出 JSON 的字段完整性与引用链:

```
import json
d = json.load(open("my-context.json", encoding="utf-8"))
msgs = d["conversations"][0]["messages"]
print([m["id"] for m in msgs]) # ['n001'..'n005']
print([m["id"] for m in msgs if "reply_to" in m]) # ['n003'] 引用链在
```

目的确认 `id/sender/timestamp/type/content/is_outgoing/reply_to` 六要素齐全——这才是"Agent 能引用原文"的合同 | 检测

✅ 预期结果 (实测): 六字段全在; `is_outgoing` 布尔正确; 引用链无损。结论: 数据层确实"可信、可追溯", 配得上它 README 里"先做好地基"的定位

学习结论与三方互链

★ 一句话带走 + 知识库互链

微信 AI 记忆库 = 个人记忆流水线的"进料口": 它解决的是"上下文原料从哪来"——用双数据源 + 适配器隔离的架构, 把最黑的盒子 (微信本地库) 变成最白的格式 (规范 JSON)。智能检索、Agent 接口它都还没做, 也不急着做。

vault 互链: **[[CleanRoom只读读取器]]** (同目标不同路线的读库之争) · **[[确定性信号引擎]]** (下游加工车间) · **[[跨工具个人记忆层]]** (下游仓库) · **[[记忆四层模型]]** / **[[混合检索与RRF融合]]** (记忆的存法与取法) · **[[事实与判断分离]]** (它只产事实层, 判断留给上层) · **[[护栏模式Guardrails]]** (ConversationKind 拒收非法值的确定性校验)。